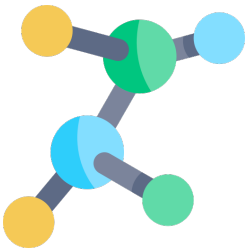


Interface d'Analyse Moléculaire

Analyseur moléculaire multi-bases de données

Auteur :
Noé Gabillaud

Superviseur :
François Bondu



mai/juin 2026

Documentation Technique

Table des matières

1 Introduction **2**

2 Démarrage Rapide **3**
2.1 Prérequis . . . 3
2.2 Installation des dépendances . . . 3
2.3 Lancement de l'interface . . . 3
2.3.1 Résultats via l'interface HTML . . . 3
2.3.2 Résultats via l'interpréteur Python . . . 6

3 Architecture du Projet **7**
3.1 Organisation des fichiers . . . 7
3.2 Flux de données . . . 7
3.3 Rôle des parsers . . . 8

4 Guide des Modules **9**
4.1 Orchestration logicielle : le module `app.py` . . . 9
4.2 Les parsers : extraction des données . . . 10
4.2.1 Parser PubChem . . . 10
4.2.2 Parser Astroquery : points de résonance spectrale . . . 11
4.2.3 Parser Thermo-Chemical : données thermodynamiques . . . 12
4.2.4 Parser NIST : extraction par scraping . . . 12
4.2.5 Parser mVOC : automatisation navigateur . . . 13
4.2.6 Parser Sander : constantes de Henry . . . 14
4.3 Outils de calcul et utilitaires . . . 14
4.4 Visualisation et génération de graphiques . . . 15
4.5 Gestion de l'interface : templates HTML . . . 16

5 Fonctionnalités de l'interface HTML **17**
5.1 Mode Light/Dark . . . 17
5.2 Export/Print . . . 17
5.3 Exit . . . 17
5.4 Liens et copie . . . 17
5.5 Codes H . . . 17
5.6 Export en CSV . . . 18

Chapitre 1

Introduction

Ceci est la documentation de l'Interface d'Analyse Moléculaire. Ce document a pour but de présenter l'architecture et le fonctionnement de cet outil, facilitant ainsi sa prise en main par les utilisateurs.

Le banc "BacTera" de l'Institut FOTON a pour but de mesurer en spectroscopie millimétrique/submillimétrique les métabolites volatiles des microbes, et en particulier des microbes des sols agricoles, en lien avec le laboratoire ECOBIO. À terme cet instrument pourra contribuer à évaluer le fonctionnement biologique des sols. Dans le cadre de ce projet, des informations sur les molécules étudiées sont nécessaires. Or, les informations nécessaires se trouvent sur différentes bases de données. Le but de l'Interface d'Analyse Moléculaire est donc de réunir les informations des différentes bases de données en une seule interface.

Les données recherchées sont :
- Des informations générales sur l'identité de la molécule
- La masse molaire
- Le logP
- La température d'ébullition
- La pression de vapeur
- Le moment dipolaire
- Les enthalpies de vaporisation et de fusion
- La constante d'Henry
- Des informations de sécurité
- Le spectre sur la plage de fréquences et la résolution choisies
- La fréquence du pic et la largeur Doppler

Les bases de données que l'on utilise sont :
- PubChem
- NIST
- mVOC
- Sander
- CDMS, JPL et HITRAN via la librairie Astroquery
- Thermo-Chemical

Chapitre 2

Démarrage Rapide

2.1 Prérequis

Avant de commencer, assurez-vous que votre environnement de travail est correctement configuré. Le projet nécessite les éléments suivants :

— **Python** installé sur votre machine.
— **Spyder** (ou tout autre IDE Python).
— Les bibliothèques listées dans le fichier `requirements.txt`.

2.2 Installation des dépendances

L'installation des bibliothèques se fait en exécutant le script `installation.py` dans un interpréteur Python. Autrement, pour installer automatiquement toutes les bibliothèques nécessaires, ouvrez un terminal dans le dossier du projet et exécutez la commande suivante :

```
pip install -r requirements.txt
```

Listing 2.1 – Installation des packages

2.3 Lancement de l'interface

Une fois les dépendances installées, le lancement se fait de trois manières différentes : via le fichier `.exe` "Windows Interface Launcher", uniquement pour le système d'exploitation Windows, ou via le script `launcher.py`, pour les autres systèmes d'exploitation, afin d'obtenir les résultats sur une interface HTML (manière privilégiée). On peut aussi utiliser le script `main.py` afin d'obtenir les résultats dans l'interpréteur Python.

2.3.1 Résultats via l'interface HTML

Avec le fichier `.exe` :

Pour obtenir l'interface :

1. Ouvrez le fichier `.exe` "Windows Interface Launcher".
2. Le système lancera automatiquement `app.py`.

Avec le fichier `launcher.py` :

Pour obtenir l'interface :

1. Ouvrez le fichier `launcher.py` dans votre éditeur.
2. Exécutez le script.
3. Le système lancera automatiquement `app.py`.

Après le lancement, votre navigateur par défaut devrait s'ouvrir automatiquement sur l'adresse `http://127.0.0.1:5000/`. Si la page s'affiche, votre installation est terminée et fonctionnelle. Il suffit désormais de chercher la molécule (nom, formule ou PubChem CID) souhaitée dans le moteur de recherche, avec les paramètres voulus.

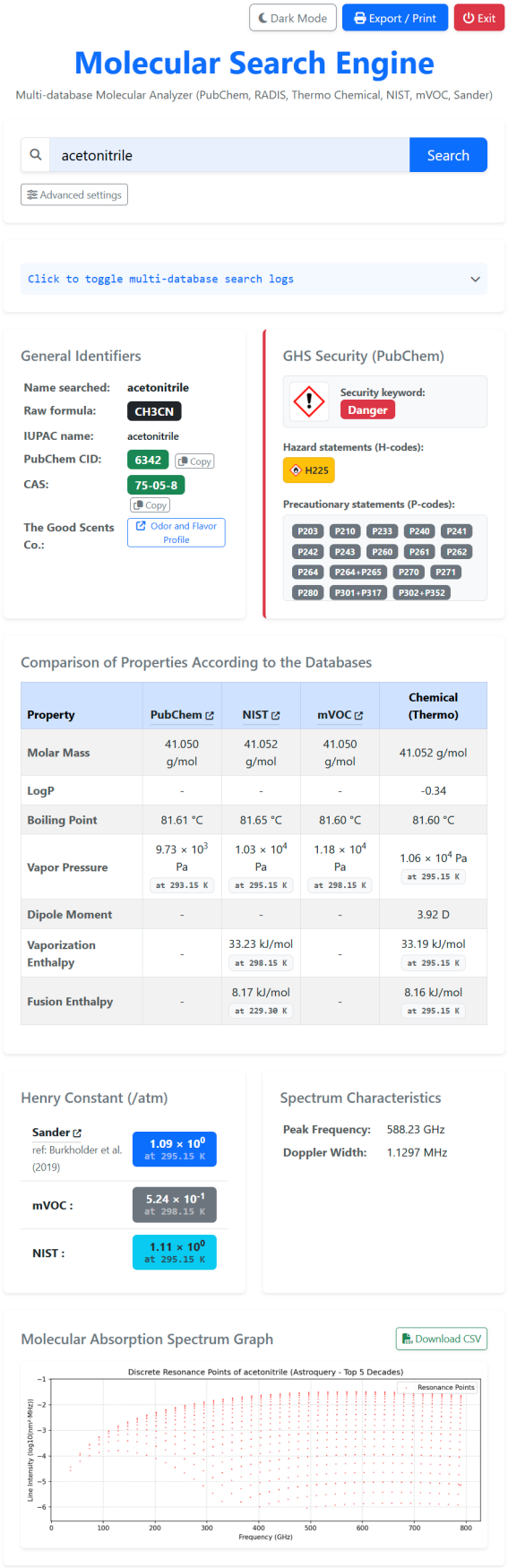


FIGURE 2.1 – Exemple : interface obtenue pour l'acétonitrile

2.3.2 Résultats via l'interpréteur Python

Pour obtenir les données :

1. Ouvrez le fichier `main.py` dans votre éditeur.
2. Entrez le nom ou la formule de la molécule souhaitée à la ligne 13.
3. Réglez éventuellement les paramètres de recherche associés.
4. Exécutez le script.
5. Les résultats apparaîtront dans l'interpréteur Python.

```
[PubChem] OK : CAS=75-05-8 CID=6342
[Astroquery]: CH3CN via CDMS (Discrete)
[Chemical]: OK
[NIST]: OK
[mVOC]: OK
[Sander]: OK

=== Formula ===
C2H3N

=== IUPAC Name ===
acetonitrile

=== SMILES ===
CC#N

=== Molar Mass ===
PubChem = 41.05 g/mol, Chemical = 41.052 g/mol, NIST = 41.052 g/mol, mVOC = 41.05
     g/mol

=== logP ===
PubChem = None, Chemical = -0.34, mVOC = 0.0

=== Boiling Point ===
PubChem = 81.61 °C, Chemical = 81.6 °C NIST = 81.65 °C, mVOC = 81.6 °C

=== Vapor Pressure ===
PubChem = 11839.03 Pa at 298.15 K, Chemical = 12146.63 Pa at 298.15 K, NIST =
    11795.09 Pa at 298.15 K, mVOC = 11839.03 Pa at 298.15 K

=== Dipole Moment ===
PubChem = None D, Chemical = 3.92 D, NIST = None D

=== Vaporization Enthalpy ===
PubChem = None kJ/mol at None K, Chemical = 33.05 kJ/mol NIST = 33.23 kJ/mol at
    298.15 K

=== Fusion Enthalpy ===
PubChem = None kJ/mol at None K, Chemical = 8.16 kJ/mol NIST = 8.17 kJ/mol at
    229.3 K

=== Henry Constant ===
HenryConstant(Sander = 0.95 /atm at 298.15 K, mVOC = 0.52 /atm at 298.15 K, NIST
    = 0.97 /atm at 298.15 K

=== Security ===
Signal = ['Danger'], H = ['H225'], P = ['P203', 'P210', 'P233', 'P240', 'P241', '
    P242', 'P243', 'P260', 'P261', 'P262', 'P264', 'P264+P265', 'P270', 'P271', '
    P280', 'P301+P317', 'P302+P352', 'P303+P361+P353', 'P304+P340', 'P305+P351+
    P338', 'P305+P354+P338', 'P308+P316', 'P316', 'P317', 'P318', 'P319', 'P321',
     'P330', 'P337+P317', 'P361+P364', 'P362+P364', 'P370+P378', 'P403+P235', '
    P405', 'P501']

=== Spectrum ===
Peak frequency = 588.46 GHz, Doppler width = 1.136 MHz
```

Listing 2.2 – Exemple : texte obtenu pour l'acétonitrile

NB : Le spectre est également tracé.

Chapitre 3

Architecture du Projet

Ce chapitre présente l'organisation générale du code source ainsi que le fonctionnement de l'application.

3.1 Organisation des fichiers

Chaque fichier du projet possède un rôle spécifique :

- `Windows Interface Launcher.exe` : point d'entrée principal du programme pour Windows. Il vérifie l'environnement Python avant de lancer le serveur.
- `Interface_Launcher_via_Python.py` : point d'entrée principal universel du programme. Il vérifie également l'environnement Python avant de lancer le serveur.
- `app.py` : contrôleur principal basé sur Flask. Il gère les échanges entre l'utilisateur et les différents parsers.
- `main.py` : version console du programme permettant d'exécuter les parsers sans interface web.
- `solver.py` : coordonne les différents parsers et centralise les données récupérées pour une recherche via `main.py`.
- `/templates` : contient les fichiers HTML définissant la structure des pages web.
- `/static` : regroupe les ressources utilisées par l'interface (images, styles, etc.).

3.2 Flux de données

Le fonctionnement de l'application se déroule en plusieurs étapes :

1. L'utilisateur renseigne une molécule via l'interface web ou directement dans le programme Python.
2. Le contrôleur (`app.py` ou `solver.py`) transmet la requête aux différents parsers.
3. Chaque parser interroge sa base de données, extrait les informations utiles puis les normalise.
4. Les données récupérées sont ensuite affichées dans l'interface web ou dans la console Python.

3.3 Rôle des parsers

Les parsers assurent la communication entre l'application et les bases de données externes telles que PubChem, CDMS, JPL, HITRAN, Chemical, NIST, mVOC ou Sander.

Chaque parser possède trois fonctions principales :

— **Requête** : envoyer une demande à une base de données via une API ou du scraping.

— **Normalisation** : convertir les données récupérées dans un format commun afin d'uniformiser les unités et les structures.

— **Gestion d'erreurs** : éviter les interruptions du programme lorsqu'une donnée est absente ou qu'une base ne répond pas.

Cette organisation permet d'isoler chaque source de données. Ainsi, si une base de données change de fonctionnement, seul le parser correspondant doit être modifié sans impacter le reste du projet.

Chapitre 4

Guide des Modules

Ce chapitre décrit l'organisation interne des différents composants utilisés pour l'extraction, le traitement et l'affichage des données moléculaires.

4.1 Orchestration logicielle : le module `app.py`

Le fichier `app.py` joue le rôle de point central de l'application. Contrairement aux modules spécialisés dans l'extraction de données, il coordonne l'ensemble du flux de traitement, depuis la réception de la requête utilisateur jusqu'à l'affichage des résultats sur la page HTML.

Gestion du cycle de vie des requêtes

Le module utilise le framework `Flask` pour définir les routes de l'application. Son rôle est d'enchaîner les différentes opérations nécessaires au traitement d'une molécule :

1. **Réception et validation :** récupération de l'entrée utilisateur (nom de molécule ou formule) puis initialisation de l'objet `molecule`.
2. **Appel des services :** exécution des différents parsers (PubChem, NIST, mVOC, etc.) en fonction des identifiants disponibles (CAS, CID).
3. **Calculs physiques :** lancement des outils de normalisation et des fonctions d'extrapolation, notamment pour la pression de vapeur avec les paramètres d'Antoine ou la loi de Henry.
4. **Rendu visuel :** transmission de l'objet `molecule` enrichi au moteur de template `Jinja2` afin de générer la page de résultats.

Robustesse et gestion des exceptions

Afin qu'une erreur provenant d'une source de données n'interrompe pas toute l'application, `app.py` intègre une gestion globale des exceptions :

— **Isolation des erreurs :** chaque appel de service est encapsulé dans un bloc `try/except`. Lorsqu'une donnée est indisponible, la valeur est laissée vide sans empêcher l'affichage des autres propriétés.

— **Suivi des opérations :** le module génère des logs permettant de suivre les différentes étapes d'exécution et de faciliter le débogage (barre d'avancement sur la page HTML).

Architecture orientée objet

Le module repose sur une structure de données unifiée basée sur la classe `molecule`. Toutes les sources de données — API, scraping web ou calculs locaux — alimentent les mêmes attributs normalisés.

Cette organisation simplifie l'ajout de nouvelles sources : il suffit de développer un nouveau parser puis de l'intégrer dans la séquence d'exécution de `app.py`.

4.2 Les parsers : extraction des données

Chaque fichier du dossier `/parsers` est dédié à une source de données.

4.2.1 Parser PubChem

Le module `pubchem_parser.py` récupère les informations moléculaires depuis la base de données PubChem. Les données retournées par l'API étant fortement imbriquées, le parser utilise une méthode d'exploration adaptée à la structure JSON fournie par PubChem.

Extraction récursive des propriétés

L'API `PUG View` retourne un fichier JSON dont l'organisation varie selon les molécules. La fonction `extract_pubchem_property` parcourt récursivement cette structure :

— recherche des sections contenant la clé `TOCHeading` correspondant à la propriété demandée ;

— extraction des valeurs présentes dans `StringWithMarkup`.

Cette méthode permet de récupérer des propriétés comme le point d'ébullition, la pression de vapeur ou les données de sécurité même lorsque leur position dans le JSON change d'une molécule à l'autre.

Nettoyage et standardisation

Les données extraites étant hétérogènes, le parser applique plusieurs traitements avant leur utilisation :

1. **Normalisation via expressions régulières :** les fonctions de nettoyage convertissent les notations scientifiques textuelles en nombres exploitables par Python.
2. **Sélection des conditions expérimentales :** pour les propriétés dépendantes de la température, la méthode `get_closest_to_temperature` choisit automatiquement la valeur expérimentale la plus proche de la température demandée.

Extraction des données GHS

Le parser récupère également les informations de sécurité chimique via la fonction `get_ghs` :

— détection des mentions d'avertissement telles que “Danger” ou “Attention” ;

— extraction des codes H et P à l'aide d'expressions régulières.

Les données obtenues sont ensuite réutilisées dans l'interface afin d'afficher les pictogrammes et avertissements correspondants.

4.2.2 Parser Astroquery : points de résonance spectrale

Le module `astroquery_parser.py` est chargé de récupérer les raies et les points de résonance discrets du spectre d'une molécule donnée. Pour ce faire, il interroge à distance des catalogues spectroscopiques de référence via les services d'`astroquery`, à savoir `CDMS`, `JPL` et `HITRAN`.

Résolution des molécules

L'un des principaux défis consiste à faire correspondre le nom ou la formule fournis par l'utilisateur avec les conventions et identifiants stricts requis par les serveurs distants.

Le parser effectue pour cela plusieurs étapes de vérification et de conversion :

— normalisation des formules chimiques avec la fonction `normalize_formula` qui décompose et compte le nombre d'atomes de la molécule ;

— récupération automatique de la formule brute via l'API PubChem (`pubchempy`) si l'identifiant initial saisi est un nom commun ;

— mise en correspondance avec le dictionnaire des molécules cibles afin d'associer la bonne clé numérique ou le bon *tag* de recherche.

Stratégie d'interrogation et de repli

Lorsqu'une recherche est initiée dans la bande de fréquence définie, le parser applique une stratégie d'interrogation séquentielle pour garantir l'obtention de données, même si certains services sont indisponibles :

1. Tentative de récupération des raies discrètes auprès de la base `CDMS` (Cologne Database for Molecular Spectroscopy) ;
2. En cas d'échec ou d'absence de lignes, bascule automatique vers le catalogue du `JPL` (Jet Propulsion Laboratory) ;
3. En dernier recours, interrogation des tables en ligne d'`HITRAN`.

Chaque requête réseau est encapsulée dans un gestionnaire de temps (`run_with_timeout`) limitant l'exécution à 25 secondes, protégeant ainsi l'application contre les latences excessives ou les pannes des serveurs distants.

Traitements physiques et calcul Doppler

Une fois le jeu de raies discrètes récupéré avec succès, le parser applique les transformations physiques suivantes :

— conversion des fréquences spectrales en gigahertz (GHz) et application d'une correction thermique sur l'intensité des raies (`log_ints`) si la température demandée s'éloigne de la température de référence du catalogue ;

— identification de la raie de résonance principale (le pic d'intensité maximale) au sein de la bande de fréquence ;

— calcul de la largeur Doppler à mi-hauteur avec `calculate_doppler_width` au niveau de ce pic principal. Ce calcul dépend de la température et de la masse molaire de la molécule (extraite des bases NIST ou PubChem) selon la formule :

$$\Delta\nu = \frac{\nu_{\text{pic}}}{c}\sqrt{\frac{8k_B T \ln 2}{m}}$$

avec m la masse d'une molécule et ν_{pic} la fréquence de la résonance maximale.

Cette organisation permet d'isoler rapidement les fréquences critiques d'une molécule de manière stable, légère et sans dépendance à des bases de données locales volumineuses.

4.2.3 Parser Thermo-Chemical : données thermodynamiques

Le module `chemical_parser.py` complète les propriétés thermodynamiques à l'aide de la bibliothèque `thermo`.

Recherche via le numéro CAS

Le parser utilise le numéro `CAS` obtenu depuis PubChem afin d'interroger la base interne de la bibliothèque `thermo`. Cette méthode limite les ambiguïtés liées aux noms chimiques.

Traitement des propriétés

Les propriétés récupérées sont directement associées aux attributs de l'objet `molecule` :
— Conversion des enthalpies de vaporisation et de fusion en kJ/mol ;
— Conversion des températures de Kelvin vers degrés Celsius ;
— Calcul de la pression de vapeur saturante à la température définie par l'utilisateur.

Gestion des erreurs

Lorsque la molécule n'est pas présente dans la bibliothèque `thermo` ou que le numéro CAS est invalide, le parser intercepte l'exception sans interrompre le reste du traitement.

4.2.4 Parser NIST : extraction par scraping

Le module `nist_parser.py` complète les données thermodynamiques en interrogeant le NIST Chemistry WebBook. Cette base ne proposant pas d'API directe, les informations sont récupérées par analyse du code HTML.

Navigation et extraction

L'extraction suit plusieurs étapes :
— Envoi de requêtes avec des paramètres adaptés aux données recherchées ;
— Analyse du HTML avec `BeautifulSoup` ;
— Reconstruction des tableaux de propriétés à partir des balises `tr` et `td` ;
— Extraction des valeurs numériques à l'aide d'expressions régulières ;
— Filtrage et sélection des données : pour garantir la pertinence des données, le parser isole en priorité les coefficients valides dans la plage de température de l'étude ou, à défaut, extrait la donnée correspondant à la température la plus proche disponible dans les tables du NIST.

Calculs Physiques

Le parser peut également calculer certaines propriétés :

1. **Équation d'Antoine :** calcul de la pression de vapeur saturante lorsque les coefficients A, B et C sont disponibles par : $P = 10^{A - \frac{B}{T+C}}$ avec T la température ;

2. **Constante de Henry :** extrapolation de la constante à la température utilisateur à partir des coefficients fournis par le NIST par : $H = H_0 e^{\texttt{dlnHs/dT}(\frac{1}{T} - \frac{1}{298.15K})}$

Gestion des imprévus

Comme tout système de scraping, ce module dépend fortement de la structure du site. Si une page ou un tableau est absent, les valeurs sont laissées à `None` afin de ne pas interrompre le traitement global.

4.2.5 Parser mVOC : automatisation navigateur

Le module `mvoc_parser.py` extrait des données physico-chimiques et écologiques depuis la base de données mVOC. Le site reposant sur une interface dynamique, le parser automatise la navigation pour soumettre les formulaires et capturer le contenu généré.

Isolation et automatisation avec Playwright

Pour interagir avec l'interface, le parser s'appuie sur la bibliothèque `Playwright` en mode *headless* (sans interface graphique). L'architecture logicielle a été conçue pour garantir une compatibilité multiplateforme et une stabilité maximale :

— **Gestion de l'Event Loop Windows :** Sur plateforme `win32`, le parser force l'utilisation de `WindowsSelectorEventLoopPolicy` pour supporter correctement les processus asynchrones sous-jacents.

— **Isolation par Thread dédié :** Contrairement à une exécution par processus lourd, le moteur de navigation est encapsulé dans un thread indépendant (`_thread_worker`). Cette isolation est indispensable pour éviter les conflits majeurs liés à la boucle d'événements `asyncio` de l'application principale ou des environnements de développement comme Spyder.

Le flux d'automatisation de la fonction de récupération (`_fetch_mvoc_html`) effectue les actions suivantes : ouverture du navigateur Chromium, navigation vers la page de recherche, saisie du CID PubChem (`compound_id`), soumission du formulaire et interception sécurisée du code HTML final transféré via un dictionnaire partagé (`container`).

Extraction avancée des données

Une fois le code HTML récupéré, la fonction `fill_from_mvoc` orchestre l'extraction des données à l'aide de `BeautifulSoup` combiné à des expressions régulières ciblées :

1. **Tableau des caractéristiques globales :** Le parser parcourt les balises `tr` pour récupérer des données clés telles que le *LogP*, le nom IUPAC, le code SMILES, la masse molaire (*Molweight*), ainsi que l'origine biologique (*in_ organism*).
2. **Constante de Henry :** Le code recherche le mot-clé « henry »et applique des motifs `regex` spécifiques capables de capturer la valeur, qu'elle soit écrite sous forme de notation scientifique ($a \times 10^{-b}$) ou d'estimation textuelle brute.
3. **Points d'ébullition et pression de vapeur :** Le parser scanne l'ensemble des tables HTML. Il extrait le point d'ébullition en degrés Celsius et isole les données de pression de vapeur saturante en identifiant dynamiquement l'unité originale (`mmHg`, `Pa`, `atm`, `kPa`) ainsi que la température associée, convertie d'office en Kelvin.

Normalisation et Intégration

Les données brutes extraites subissent un traitement de nettoyage et d'uniformisation avant d'être intégrées. Le module utilise les fonctions utilitaires du noyau (`core.tools`) :

- `preprocess` est appliquée sur les chaînes numériques pour standardiser les formats textuels ;
- `convert_value` gère les changements d'unités physiques (par exemple, la conversion de la pression de vapeur de `mmHg` vers l'unité standard de l'application, le Pascal `Pa`).

À la fin du processus, les valeurs nettoyées sont directement injectées dans les sous-attributs dédiés de l'objet `molecule` (`mol.molar_mass.mvoc`, `mol.logp.mvoc`, `mol.boilingtemp.mvoc`, etc.), assurant une traçabilité totale de la source d'information `mvoc` au sein du système.

4.2.6 Parser Sander : constantes de Henry

Le module `sander_parser.py` est dédié à l'extraction des constantes de Henry depuis la base Sander.

Recherche via le numéro CAS

Le parser utilise le numéro CAS comme identifiant principal afin d'éviter les ambiguïtés entre molécules proches.

Sélection des données

Plusieurs entrées pouvant être disponibles pour une même molécule, le parser applique une logique de sélection :

- priorité aux données contenant le coefficient `dlnHs/dT` et calcul de la constante d'Henry H à la température T par : $H = H_0 e^{\texttt{dlnHs/dT}(\frac{1}{T} - \frac{1}{298.15K})}$;
- utilisation d'une valeur de référence lorsque ce coefficient n'est pas disponible.

Nettoyage et conversion

Les données récupérées sont nettoyées puis converties :

1. suppression des caractères spéciaux présents dans le HTML ;
2. calcul de la constante d'Henry à la température souhaitée si le coefficient `dlnHs/dT` est disponible ;
3. conversion des constantes de Henry vers des unités homogènes (atm^{-1}).

Ce module est principalement utilisé pour les calculs liés à la solubilité des composés gazeux.

4.3 Outils de calcul et utilitaires

Le fichier `tools.py` regroupe les fonctions utilitaires utilisées par les différents parsers.

- **Conversions d'unités :** pression, température, énergie, enthalpie, etc.
- **Validation des données :** vérification des valeurs avant leur affichage.

Le module sert principalement à uniformiser les données provenant de sources différentes.

Prétraitement des données

La fonction `preprocess` nettoie les chaînes de caractères avant conversion :
— remplacement des exposants Unicode ;
— standardisation de la notation scientifique ;
— correction des caractères typographiques.

Conversion des unités

Le module utilise la bibliothèque `pint` pour effectuer les conversions :

1. détection automatique des unités présentes dans le texte ;
2. conversion vers des unités communes ;
3. gestion des lignes contenant plusieurs couples valeur/unité.

Fonctions physiques

Certaines fonctions implémentent directement des relations physiques :
— extrapolation de la constante de Henry avec `henry_at_T` ;
— formatage des nombres avec `clean_float`.
Grâce à ces outils, les parsers peuvent se concentrer sur l'extraction des données tandis que `tools.py` assure leur homogénéisation.

4.4 Visualisation et génération de graphiques

Le module de génération graphique transforme les données spectrales en représentations exploitables dans l'interface web.

Architecture de rendu

Le système repose sur `matplotlib`, configuré avec le moteur *Agg*. Cette configuration permet de générer les figures côté serveur sans interface graphique locale.

Fonction de génération

La fonction `plot_absorption_spectrum` suit plusieurs étapes :

1. configuration des axes et du style du graphique ;
2. génération de l'image dans un tampon mémoire ;
3. encodage de l'image en Base64 ;
4. fermeture systématique de la figure avec `plt.close()`.

Cette méthode évite la création de fichiers temporaires et réduit les accès disque.

Gestion des erreurs

Si aucune donnée spectrale n'est disponible, la fonction renvoie simplement une valeur vide afin d'éviter une erreur côté serveur.

4.5 Gestion de l'interface : templates HTML

Le dossier `/templates` contient les pages HTML utilisées par Flask.
— **Jinja2 :** moteur de template utilisé pour injecter dynamiquement les données dans les pages HTML ;
— **Bootstrap ou CSS personnalisé :** gestion de la mise en page et adaptation aux différentes tailles d'écran.

Organisation de la page de résultats

La page `results.html` regroupe les informations en plusieurs sections :
— **Bloc identificateurs :** nom, formule brute, CID PubChem, représentation SMILES mais également lien vers la page The Good Scents Company ;
— **Bloc spectre :** affichage des points de résonance ;
— **Tableau comparatif :** regroupement des propriétés issues des différentes sources ;
— **Autres informations :** affichage des constantes d'Henry et des informations de sécurité.

Visualisation et sécurité

Le template intègre plusieurs éléments dynamiques :

1. Affichage des pictogrammes GHS à partir des codes H et P ;
2. Intégration directe du spectre sous forme d'image encodée en Base64.

Expérience utilisateur

L'interface applique plusieurs choix de présentation :
— utilisation de la notation scientifique pour certaines valeurs ;
— ajout de liens vers les bases de données externes ;
— affichage de badges colorés selon la disponibilité des données.

L'ensemble permet de présenter des données provenant de sources variées dans une interface homogène et exploitable pour l'utilisateur.

Chapitre 5

Fonctionnalités de l'interface HTML

Ce chapitre décrit les différentes fonctionnalités de la page HTML.

5.1 Mode Light/Dark

Il a été décidé d'ajouter un bouton `Light/Dark Mode` qui change la couleur de la page. Les couleurs de police et du tableau s'adaptent. Les couleurs du spectre sont également inversées afin d'obtenir un résultat cohérent avec le reste.

5.2 Export/Print

Le bouton `Export/Print` permet d'exporter les résultats de la page en PDF ou bien de les imprimer. Il est conseillé de passer en `Light Mode` avant d'exporter ou d'imprimer la page.

5.3 Exit

Le bouton Exit permet d'arrêter proprement le programme, sans passer par la fenêtre `.exe` ou l'interpréteur Python.

5.4 Liens et copie

En cliquant sur le nom des bases de données dans le tableau de résultats, la page de la molécule sur la base de données associée est ouverte dans un nouvel onglet. Il y a également des boutons pour copier le CID et le CAS de la molécule. Il est nécessaire d'avoir le CID de la molécule pour la rechercher sur mVOC.

5.5 Codes H

Chaque code H est associé à un pictogramme et une explication. Il est possible de connaître la signification du code H en glissant sa souris dessus.

5.6 Export en CSV

Le bouton en haut à droite du spectre permet de télécharger un fichier `.csv` comportant les points du spectre sur la plage de fréquences et la résolution choisies lors de la recherche de la molécule. Il peut y avoir un certain temps entre le clic sur le bouton et le téléchargement, il s'explique par la potentielle grande quantité de données à télécharger.